광운대학교 로봇게임단 로빛

과제보고서

미션 5 단계 증거 밑 기타

학 번	2024402035
이 름	장경민
제 출 일	2026. 11. 06.

목차

1. 들어가며 2

2. 의도하지 않은 방향의 과제 해결 2

3. robit_blackbox report 4

4. 단계 1 4

4.1 개요 4
4.2 사용한 스크립트와 해결과정 4
4.3 시행 착오 5
4.3.1 스크립트 오류 5
4.3.2 토픽 전송 오류 5

5. 단계 2 5

5.1 개요 5
5.2 해결과정 5
5.3 시행착오 7
5.3.1 QoS 설정 오류 7
5.3.2 KeepLast 설정 7

6. 단계 3 7

6.1 개요 7
6.2 해결과정 8
6.2.1 도메인 ID 9
6.2.2 peers 설정 및 AllowMulticast 설정 9
6.2.3 Verbosity 9
6.2.4 최종 결과 10
6.3 시행 착오 10

7. 단계 4 11

7.1 개요 11
7.2 해결과정 11

8. 단계 5 12

8.1 개요 12
8.2 도메인 ID 와 포트번호 12
8.3 궁금증 14

1. 들어가며

이 과제는 강의 시간에 배운 ROS 의 통신과 관련된 미들웨어를 이용하여 몇 가지 간단한 과제를 해결하는 것임.

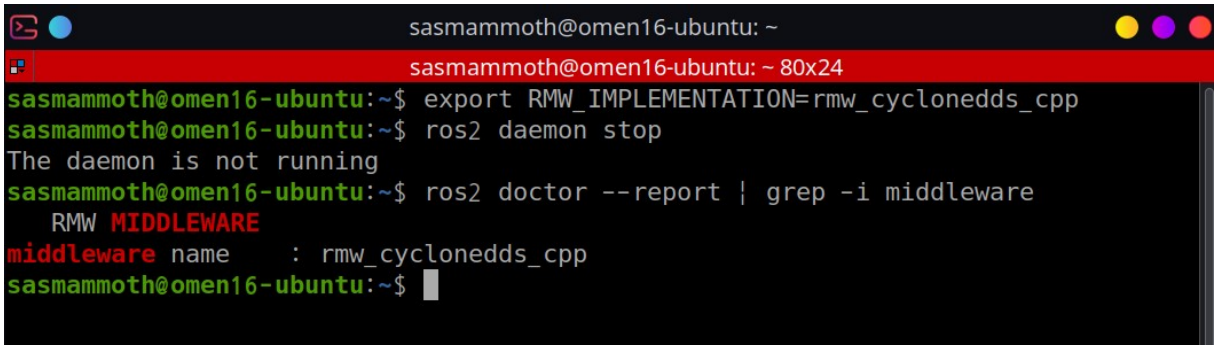


그림 1: 미들웨어를 cyclonedds 로 바꾼 화면이다

본격적인 시작에 앞서서 ros2-jazzy 용 cyclonedds 를 설치하여 터미널에 한번 적용하여본 사진이다.

2. 의도하지 않은 방향의 과제 해결

과제 1 을 해결하기 위해서 도메인을 스캔하여 토픽이름을 찾고 그거에 맞는 도메인 id 를 찾은뒤에 그거에 토픽에 나와있는대로 알맞은 챌린지 단어를 넣어서 문제를 해결하라고 되어있다. 도메인을 스캔한다는 것을 제대로 이해하지 못하여서 도메인 ID 를 찾을 방법을 생각하다가 강의자료 마지막에 명령어 치트시트의 ss -ulpn 과 도메인 id 생성 규칙(RTPS)를 조합하면 도메인 id 를 얻을 수 있을 거 같다는 생각이 들었다.

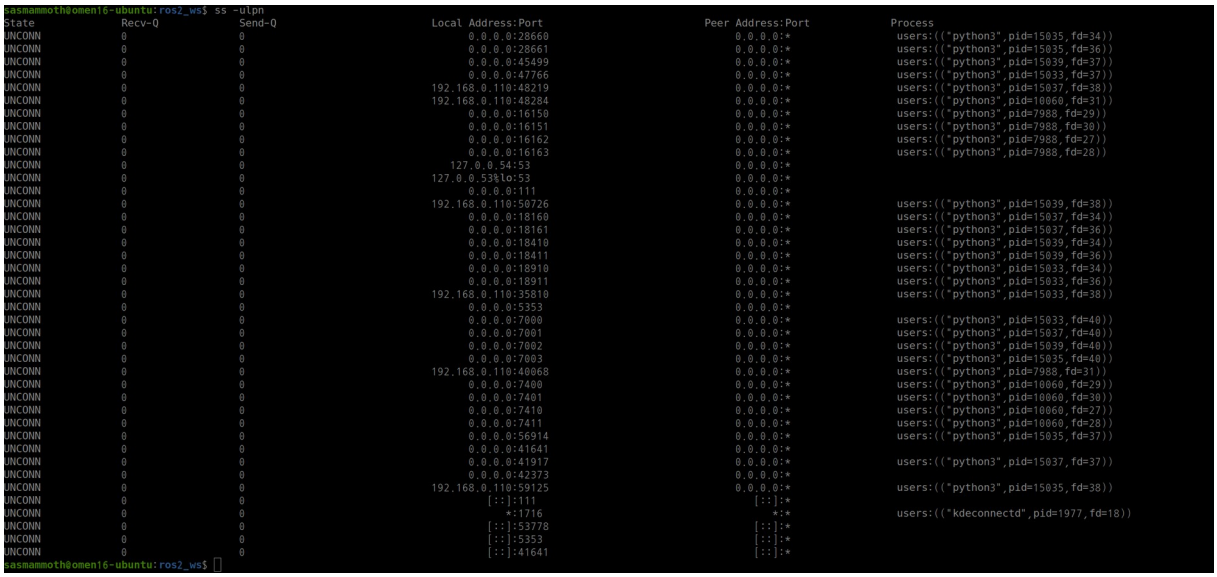


그림 2: 열린 UDP 포트 확인

ss -ulpn 을 이용하여 열린 udp 포트를 확인하고 여기서 포트번호가 RTPS 규칙에 맞는 포트를 찾았다. RTPS 규칙에서 offset 은 0,10, 1, 11 이므로 이 만큼 차이가 나고 같은 PID 를 가지고 있는 특징적인 포트번호 몇 가지를 찾아서 규칙 공식에 대입하였다.

표 1: 포트번호 부여 공식

$$\text{포트} = PB + DG \times \text{domainId} + PG \times \text{participantId} + \text{offset}$$

$$PB = 7400 \cdot DG = 250 \cdot PG = 2$$

표 2: 특징적인 몇가지 포트번호 계산

16150 = 7400 + x * 250 + 2 * ? + 0 (x = 35, ? = 0)
: 없음

18160 = 7400 + x * 250 + 2 * ? + 0 (x:id = 43, ? = 5)
: 단계 4

18910 = 7400 + x * 250 + 2 * ? + 0 (x:id = 46, ? = 5)
: 단계 1

28660 = 7400 + x * 250 + 2 * ? + 0 (x:id = 85, ? = 5)
: 단계 2

participantId 는 알 수 있는 방법이 없어서 숫자를 하나씩 넣어가며 x 가 정수가 될 때 까지 계산하였다.

```
sasmammoth@omen16-ubuntu:ros2_ws$ export ROS_DOMAIN_ID=43
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 topic list
/bb_ae3e/cmd
/parameter_events
/rosout
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 node list
/bb_talker4
sasmammoth@omen16-ubuntu:ros2_ws$ export ROS_DOMAIN_ID=35
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 node list
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 topic list
/parameter_events
/rosout
sasmammoth@omen16-ubuntu:ros2_ws$ export ROS_DOMAIN_ID=46
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 topic list
/bb_ae3e/ack
/bb_ae3e/telemetry
/parameter_events
/rosout
sasmammoth@omen16-ubuntu:ros2_ws$ export ROS_DOMAIN_ID=0
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 topic list
/parameter_events
/rosout
sasmammoth@omen16-ubuntu:ros2_ws$ export ROS_DOMAIN_ID=85
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 topic list
/bb_ae3e/answer
/bb_ae3e/config
/bb_ae3e/stream
/parameter_events
/rosout
sasmammoth@omen16-ubuntu:ros2_ws$
```

그림 3: 도메인 ID 를 기입하여 찾은 토픽

이를 이용해 몇 가지 도메인 ID 를 넣어서 문제에 필요한 도메인 id 를 찾았다.

3. robit_blackbox report

```
sasmammoth@omen16-ubuntu:src$ ros2 run robit_blackbox report --name "JKM"
=== ROBIT 블랙박스 제출 요약 ===
이름: JKM (id f9b3375d)
  1 단계 정찰                  통과  2026-11-05T21:52:32+09:00
  2 단계 QoS                 통과  2026-11-05T23:34:57+09:00
  3 단계 Cyclone 설정          통과  2026-11-06T01:06:38+09:00
  4 단계 고장 진단 (도전)          통과  2026-11-06T00:14:48+09:00
기록 검증: 정상
기록 해시: 7cff6fdb037ebfbc
이 출력을 보고서 맨 위에 붙여 넣으세요.
```

4. 단계 1

4.1 개요

과제 1 은 /bb_xxxx/telemetry 형식의 토픽이 돌아다니는 도메인 ID 를 찾는 것이다. 이를 위해 간단한 스크립트를 작성하여 도메인 ID 를 찾았다.

4.2 사용한 스크립트와 해결과정

AI 를 이용해 스크립트 문법에 대해 배우고, 스크립트를 돌려서 토픽을 찾았다.

```
#!/bin/bash
set -eo pipefail

echo "hello"

for i in {10..95};
do
echo "domainID ${i}"
export ROS_DOMAIN_ID=$i
ros2 topic list
echo ""
done
```

사용한 스크립트 이다. 원본은 /script/scout.sh 이다.

스크립트 실행 후 화면이다. telemetry 가 있는 도메인 id 는 46 이고 토픽명은 /bb_ae3e/telemetry 이므로 이를 수신하면 다음과 같이 나온다. CLACUER-7440 을 한번 발행하여 문제를 해결 하였다.

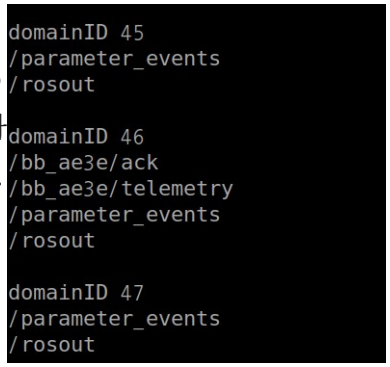


그림 4: 스크립트 실행 중 발췌

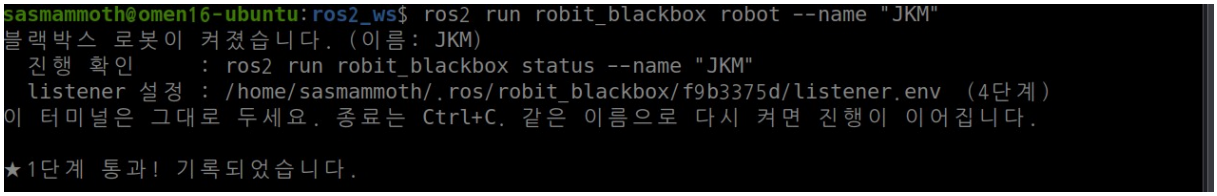


그림 5: 과제 1 해결한 모습

4.3 시행 착오

4.3.1 스크립트 오류

스크립트를 다음과 같이 작성하여 오류가 나옴

```
export ROS_DOMAIN_ID=$i:
```

:을 작성하면 명령어를 순차 실행할 때 앞에게 성공하여야 뒤에게 작동한다고 하여 :을 붙여서 작성하였다. 저렇게 붙어 있으면 ...ID=10:으로 들어가서 파이썬 내에서 정수로 변환 할 수 없어서 동작을 안한다는 것을 알게됨.

4.3.2 토픽 전송 오류

```
ros2 topic pub -1 /bb_ae3e/ack std_msgs/msg/String "GLACIER-7440"
```

처음에 다음과 같이 작성하였는데 오류가 발생함. 찾아보니 모든 토픽은 yaml 형식으로 주고 받는다는 것을 알게되어 다음과 같이 수정함

```
ros2 topic pub -1 /bb_ae3e/ack std_msgs/msg/String "{data: GLACIER-7440}"
```

5. 단계 2

5.1 개요

QoS 가 다른 토픽 2 개를 받을 수 있는 노드를 만드는 부분이다.

5.2 해결과정

```
config
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: TRANSIENT_LOCAL
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
```

```
stream
QoS profile:
  Reliability: BEST_EFFORT
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
```

위 부분은 각각 토픽의 QoS 부분을 발췌한 부분이다. 위 2 개 중 강의자료에 나온 것과 서로 다른 부분만 따로 보면 다음과 같다.

```
Config
Reliability: RELIABLE
Durability: TRANSIENT_LOCAL

stream
Reliability: BEST_EFFORT
Durability: VOLATILE
```

이를 강의 자료에 있는 QoS 조합표로 호환성에 맞는 노드를 다음과 같이 만들어서 수신 하였다.

	Reliability	Durability
sub_config	reliable (둘 다 상관 없음)	transient local (둘 다 상관 없음)
sub_stream	best effort (이거만 가능)	Volatile (이거만 가능)

```
...
EduDdsTast2Node::EduDdsTast2Node() : Node("edu_dds_task2_node")
{
  sub_config = create_subscription<std_msgs::msg::String>(
      "/bb_ae3e/config", rclcpp::QoS(rclcpp::KeepLast(1)).reliable().transient_local(),
      [this](const std_msgs::msg::String::SharedPtr msg)
      { config_collback(msg); });
  sub_stream = create_subscription<std_msgs::msg::String>(
      "/bb_ae3e/stream", rclcpp::QoS(rclcpp::KeepLast(1)).best_effort().durability_volatile(),
      [this](const std_msgs::msg::String::SharedPtr msg)
      { stream_collback(msg); });
}
...
```

```
[INFO] [1793887949.176084332] [edu_dds_task2_node]: stream: 'BB2-B word=bc54 n=8021'
[INFO] [1793887949.275705129] [edu_dds_task2_node]: stream: 'BB2-B word=bc54 n=8022'
[INFO] [1793887949.375887802] [edu_dds_task2_node]: stream: 'BB2-B word=bc54 n=8023'
[INFO] [1793887949.476074412] [edu_dds_task2_node]: stream: 'BB2-B word=bc54 n=8024'
^C[INFO] [1793887949.528444709] [rclcpp]: signal_handler(SIGINT/SIGTERM)
sasmammoth@omen16-ubuntu:src$ ros2 run edu_dds_task2 edu_dds_task2_node
[INFO] [1793887951.708148956] [edu_dds_task2_node]: config: 'BB2-A word=39b7 ¦ 스트림은 /bb_ae3e/stream ¦ 각 메시지의 word= 뒤 값 두 개를 A-B 순서로 "-" 로 이어 /bb_ae3e/answer 로 보내라 (구독 노드가 켜져 있는 동안) ¦ 다음 신호: 도메인 43의 /bb_ae3e/cmd'
[INFO] [1793887951.774912834] [edu_dds_task2_node]: stream: 'BB2-B word=bc54 n=8047'
[INFO] [1793887951.875857182] [edu_dds_task2_node]: stream: 'BB2-B word=bc54 n=8048'
[INFO] [1793887951.975767817] [edu_dds_task2_node]: stream: 'BB2-B word=bc54 n=8049'
[INFO] [1793887952.075537464] [edu_dds_task2_node]: stream: 'BB2-B word=bc54 n=8050'
[INFO] [1793887952.175582266] [edu_dds_task2_node]: stream: 'BB2-B word=bc54 n=8051'
[INFO] [1793887952.275322465] [edu_dds_task2_node]: stream: 'BB2-B word=bc54 n=8052'
[INFO] [1793887952.375025705] [edu_dds_task2_node]: stream: 'BB2-B word=bc54 n=8053'
[INFO] [1793887952.475118959] [edu_dds_task2_node]: stream: 'BB2-B word=bc54 n=8054'
```

저 노드를 실행시켜 위와 같은 결과를 얻었다. 두 word 를 조합하여 다음 신호의 도메인을 얻고 문제를 해결하였다.

```
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 topic pub -1 /bb_ae3e/answer std_msgs/msg/String "{data:
39b7-bc54}"
publisher: beginning loop
publishing #1: std_msgs.msg.String(data='39b7-bc54')

^Csasmammoth@omen16-ubuntu:ros2_ws$ ros2 run robit_blackbox robot --name "JKM"
블랙박스 로봇이 켜졌습니다. (이름: JKM)
  진행 확인      : ros2 run robit_blackbox status --name "JKM"
  listener 설정 : /home/sasmammoth/.ros/robit_blackbox/f9b3375d/listener.env (4단계)
이 터미널은 그대로 두세요. 종료는 Ctrl+C. 같은 이름으로 다시 켜면 진행이 이어집니다.
[WARN] [1793887279.672298000] [bb_s2]: New subscription discovered on topic '/bb_ae3e/stream', re
questing incompatible QoS. No messages will be sent to it. Last incompatible policy: DURABILITY
[2단계] 응답 값이 맞지 않습니다. 형식: config 의 word= 값, 하이픈, stream 의 word= 값.
[2단계] 응답 값이 맞지 않습니다. 형식: config 의 word= 값, 하이픈, stream 의 word= 값.
[2단계] 값은 맞습니다. 하지만 직접 만든 구독 노드가 두 토픽을 알맞은 QoS 로 구독 중인지 확인되지
않았습니다 (config X, stream X). 응답을 보낼 때 노드가 켜져 있어야 합니다.

★ 2단계 통과! 기록되었습니다.
```

5.3 시행착오

5.3.1 QoS 설정 오류

sub_stream 의 QoS 설정중 하나인 Durability 를 실수로 transient_local 로 설정하여 노드를 켜니까 다음과 같은 노란색 경고문이 나왔음.

> [WARN] [1793890238.167998694] [edu_dds_task2_node]: New publisher discovered on topic '/bb_ae3e/stream', offering incompatible QoS. No messages will be sent to it. Last incompatible policy: DURABILITY_QOS_POLICY

이를 보고 제대로 된 설정으로 수정함.

5.3.2 KeepLast 설정

처음 구독자를 생성할 때 QoS 중 2 개만을 사용하려 아래와 같이 만들었으나 빌드되지 않았다.

```
sub_stream = create_subscription<std_msgs::msg::String>(
      "/bb_ae3e/stream", rclcpp::QoS().best_effort().transient_local(),
      [this](const std_msgs::msg::String::SharedPtr msg)
      { stream_collback(msg); });
```

찾아보니 무조건 QoS(rclcpp::KeepLast(1)) 과 같이 Depth 를 설정해야 생성된다는 것을 알게되었다.

6. 단계 3

6.1 개요

체크박스 3 노드를 실행시키면 나오는 요구사항에 맞게 xml 파일을 만든다.

6.2 해결과정

가장 먼저 DDS 를 Cyclone 으로 하기 위해 로봇 터미널과 체크박스 3 터미널의 DDS 수정하였다.

```
^Csasmammoth@omen16-ubuntu:ros2_wsexport RMW_IMPLEMENTATION=rmw_cyclonedds_cpppp
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 daemon stop
The daemon is not running
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 doctor --report | grep -i middleware
   RMW MIDDLEWARE
middleware name    : rmw_cyclonedds_cpp
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 run robit_blackbox robot --name "JKM"
블랙박스 로봇이 켜졌습니다. (이름: JKM)
  진행 확인     : ros2 run robit_blackbox status --name "JKM"
  listener 설정 : /home/sasmammoth/.ros/robit_blackbox/f9b3375d/listener.env (4단계)
이 터미널은 그대로 두세요. 종료는 Ctrl+C. 같은 이름으로 다시 켜면 진행이 이어집니다.
[WARN] [1793891963.453722003] [bb_s2]: New subscription discovered on topic '/bb_ae3e/stream', re
questing incompatible QoS. No messages will be sent to it. Last incompatible policy: DURABILITY
```

```
sasmammoth@omen16-ubuntu: ros2_ws 96x30
sasmammoth@omen16-ubuntu:ros2_ws$ export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
ros2 daemon stop
The daemon has been stopped
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 doctor --report | grep -i middleware
   RMW MIDDLEWARE
middleware name    : rmw_cyclonedds_cpp
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 run robit_blackbox check3 --name "JKM"
[JKM] 3단계 요구사항
  - 검사는 ROS_DOMAIN_ID=64 에서 실행된다 (도구가 알아서 설정)
  - Peers 에 127.0.0.1 이 있다
  - AllowMulticast 는 false 또는 spdp
  - MaxAutoParticipantIndex 는 9
  - Tracing 의 Verbosity 를 지정하고 OutputFile 은 /tmp/bb3_f9b3375d.log

[NG] 설정 파일이 없습니다: /home/sasmammoth/cyclonedds_bb.xml
[ros2run]: Process exited with failure 1
```

그 후 체스박스 노드를 실행시켜 요구사항을 확인하였다. 확인한 요구사항은 위의 사진에 나와있다. 먼저 xml 파일을 생성하고 공식문서 예제 xml 을 가져왔다. 출처는 다음과 같다.

https://cyclonedds.io/docs/cyclonedds/latest/config/index.html

```
<?xml version="1.0" encoding="utf-8"?>
<CycloneDDS
    xmlns="https://cdds.io/config"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://cdds.io/config https://raw.githubusercontent.com/eclipse-cyclone
dds/cyclonedds/master/etc/cyclonedds.xsd"
>
    <Domain Id="any">
        <General>
            <Interfaces>
                <NetworkInterface autodetermine="true" priority="default"
                                  multicast="default" />
            </Interfaces>
            <AllowMulticast>default</AllowMulticast>
            <MaxMessageSize>65500B</MaxMessageSize>
        </General>
        <Tracing>
```

```
            <Verbosity>config</Verbosity>
            <OutputFile>
                    ${HOME}/dds/log/cdds.log.${CYCLONEDDS_PID}
            </OutputFile>
        </Tracing>
    </Domain>
</CycloneDDS>
```

6.2.1 도메인 ID

첫번째 요구사항을 해결하기 위해 도메인 ID 를 바꾸었다.

```
<Domain Id="64">
```

6.2.2 peers 설정 및 AllowMulticast 설정

강의자료와 공식문서 config_file 를 이용하여 다음과 같이 설정하였다.

출처 : https://cyclonedds.io/docs/cyclonedds/latest/config/config_file_reference.html#

```
...
<General>
    <Interfaces>
        <NetworkInterface autodetermine="true" priority="default"
                          multicast="false" />
    </Interfaces>
    <AllowMulticast>false</AllowMulticast>
    <MaxMessageSize>65500B</MaxMessageSize>
</General>
...
<Discovery>
    <Peers><Peer address="127.0.0.1"/></Peers>
    <ParticipantIndex>auto</ParticipantIndex>
</Discovery>
...
```

수정하거나 추가한 부분을 붉은색으로 추가하였다. 강의자료만을 보고는 peers 의 추가위치를 찾지 못하였으나, 공식문서를 보고 다음과 같은 항목이 있는것을 알게 되어 위치를 찾을 수 있었다.

```
//CycloneDDS/Domain/Discovery/Peers
//CycloneDDS/Domain/General/Interfaces/NetworkInterface
(기존에 있는 네트워크인터페이스의 위치와 대조하여 찾았다.)
```

6.2.3 Verbosity

로그를 이용하여 출력파일을 지정한다. 공식문서에서는 config 말고 여러가지 설정이 있으나, 딱히 로그 레벨에 관해 아무 말이 없어서 config 로 설정하였고, 로그 파일 위치만 수정하였다. 위치는 위와같이 공식문서를 검색하여 찾았다.

```
<Tracing>
    <Verbosity>config</Verbosity>
```

```
    <OutputFile>
        /tmp/bb3_f9b3375d.log
    </OutputFile>
</Tracing>
```

6.2.4 최종 결과

```
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 run robit_blackbox check3 --name "JKM"
[JKM] 3단계 요구사항
  - 검사는 ROS_DOMAIN_ID=64 에서 실행된다 (도구가 알아서 설정)
  - Peers 에 127.0.0.1 이 있다
  - AllowMulticast 는 false 또는 spdp
  - MaxAutoParticipantIndex 는 9
  - Tracing 의 Verbosity 를 지정하고 OutputFile 은 /tmp/bb3_f9b3375d.log

[OK] XML 형식이 올바르다
[OK] AllowMulticast 가 false 또는 spdp (현재 false)
[OK] Peers 에 127.0.0.1 이 있다
[OK] MaxAutoParticipantIndex = 9 (현재 9)
[OK] Tracing OutputFile = /tmp/bb3_f9b3375d.log
[OK] Tracing Verbosity 가 있다

Cyclone + 당신의 XML 로 talker/listener 를 최대 25초간 돌려 봅니다...
[OK] listener 가 talker 메시지를 받았다
[OK] Tracing 로그 파일이 생겼다

★3단계 통과! 기록되었습니다. 보고서에는 XML 과 로그에서 찾은 근거 2~3줄을 함께 넣으세요.
sasmammoth@omen16-ubuntu:ros2_ws$ ros2 run robit_blackbox robot --name "JKM"
블랙박스 로봇이 켜졌습니다. (이름: JKM)
  진행 확인      : ros2 run robit_blackbox status --name "JKM"
  listener 설정 : /home/sasmammoth/.ros/robit_blackbox/f9b3375d/listener.env  (4단계)
이 터미널은 그대로 두세요. 종료는 Ctrl+C. 같은 이름으로 다시 켜면 진행이 이어집니다.
[WARN] [1793891963.453722003] [bb_s2]: New subscription discovered on topic '/bb_ae3e/stream', re
questing incompatible QoS. No messages will be sent to it. Last incompatible policy: DURABILITY

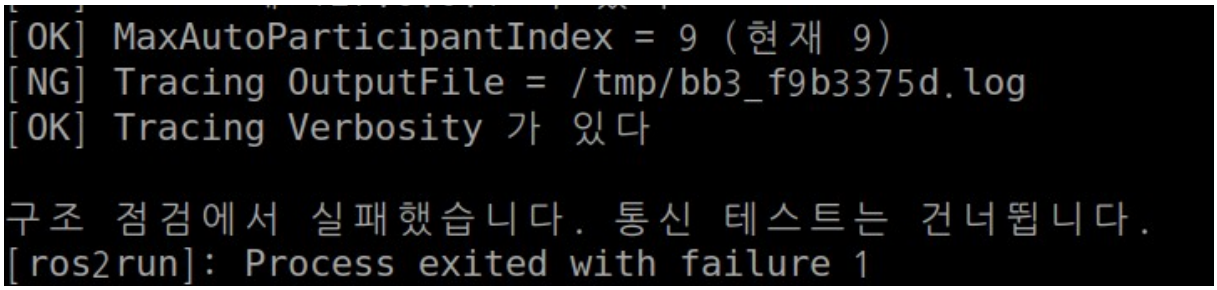
★3단계 통과! 기록되었습니다. (check 결과)
```

최종적으로 위와 같이 3 단계를 통과하였다. 최종 xml 코드는 다음과 같다.

6.3 시행 착오

나머지는 한번에 성공하였는데, OutputFile 이름을 작성할 때 다음과 같이 작성하여 몇번 실행이 안되었다.

```
<OutputFile>
    ${HOME}/tmp/bb3_f9b3375d.log
</OutputFile>
```



몇번의 변경 시행착오 후에 앞의 ${HOME}을 제거하였다. 아마 tmp 는 우분투의 임시 폴더인데, 이 위치는 사용자 계정이 아니라 OS 에 종속되어(홈디렉토리 위에 없어서) ${HOME}을 사용 안하는거 같다.

7. 단계 4

7.1 개요

로봇의 talker 을 조회하여 listener.env 의 설정을 바꾸어 listener 를 talker 와 연결한다.

7.2 해결과정

아래의 표를 체워서 해결하였다. 현재 listener 의 값은 env 파일을 그대로 작성하였다.

항목	Talker 값	listener 값	증명 명령	출력
DOMAIN	43	44	문제 3 에서 구함	-
TOPIC	/bb_ae3e/cmd	/bb_ae3e/command	ros2 topic list	/bb_ae3e/cmd ...
TYPE	std_msgs/msg/Int32	String	ros2 topic info -v /bb_ae3e/cmd	... Topic type: std_msgs/msg/Int32 ...
RELIABILITY	BEST_EFFORT	best_effort	ros2 topic info -v /bb_ae3e/cmd	... QoS profile: Reliability: BEST_EFFORT ...

따라서 잘못된 항목은 DOMAIN, TOPIC, TYPE 3 개 이고 이를 다음과 같이 수정하였다.

```
# 로봇 listener 의 설정. 이 파일을 고치면 listener 가 자동으로 다시 시작된다.
# 값 중 일부가 틀려 있다. 로봇이 보내는 신호를 조사해서 고쳐라.
LISTENER_DOMAIN=43
LISTENER_TOPIC=/bb_ae3e/cmd
LISTENER_TYPE=Int32
LISTENER_RELIABILITY=best_effort
```

```
[WARN] [1793890237.835804159] [bb_s2]: New subscription discovered on topic '/bb_ae3e/stream', re
questing incompatible QoS. No messages will be sent to it. Last incompatible policy: DURABILITY
listener.env 변경을 감지해 listener 를 다시 시작했습니다.

★4단계 통과! 기록되었습니다.
```

수정 후 저장하니 위와 같이 4 단계를 통과하였다.

8. 단계 5

8.1 개요

궁금한 것을 정하여 실험으로 답을 구하는 자유탐구 과제이다. 이미 한번 해보기도한 RTPS 공식에 관해 탐구해볼 생각이다.

8.2 도메인 ID 와 포트번호

아까와 같이 접근하려 하였는데 열린 포트가 조금 늘었다. 그래서 AI 를 이용해서 10 으로 나눠 떨어지는 것과 +1 이 존재하는 포트만을 출력하는 스크립트 코드를 만들었다.

```
#!/bin/bash
set -eo pipefail

ss -ulpn | awk '
NR == 1 { header = $0; next }
{
    n = split($4, a, ":")
    port = a[n]
    if (port !~ /^[0-9]+$/) next
    open[port] = 1                      # 열린 포트 기록
    lines[port] = lines[port] $0 "\n"    # 같은 포트가 여러 줄일 수 있어 이어 붙임
}
END {
    print header
    for (p in open) {
        p += 0                          # 문자열 키를 숫자로
        if (p % 10 == 0 && ((p + 1) in open)) {
            printf "%s%s", lines[p], lines[p + 1]
            print ""
        }
    }
}'
```

이를 돌려서 다음과 같은 항목을을 었었다.

```
sasmammoth@omen16-ubuntu:ros2_ws$ ./script/find_node_id.sh
State  Recv-Q Send-Q Local Address:Port  Peer Address:PortProcess
```

```
UNCONN 0      0            0.0.0.0:7400       0.0.0.0:*    users:(("python3",pid=49932,fd=29))
UNCONN 0      0            0.0.0.0:7401       0.0.0.0:*    users:(("python3",pid=49932,fd=30))

UNCONN 0      0            0.0.0.0:12410      0.0.0.0:*    users:(("python3",pid=63790,fd=34))
UNCONN 0      0            0.0.0.0:12411      0.0.0.0:*    users:(("python3",pid=63790,fd=36))

UNCONN 0      0            0.0.0.0:10910      0.0.0.0:*    users:(("python3",pid=63796,fd=34))
UNCONN 0      0            0.0.0.0:10911      0.0.0.0:*    users:(("python3",pid=63796,fd=36))

UNCONN 0      0            0.0.0.0:10660      0.0.0.0:*    users:(("python3",pid=63794,fd=34))
UNCONN 0      0            0.0.0.0:10661      0.0.0.0:*    users:(("python3",pid=63794,fd=36))

UNCONN 0      0            0.0.0.0:16160      0.0.0.0:*    users:(("python3",pid=63792,fd=34))
UNCONN 0      0            0.0.0.0:16161      0.0.0.0:*    users:(("python3",pid=63792,fd=36))

UNCONN 0      0            0.0.0.0:7000       0.0.0.0:*    users:(("python3",pid=63792,fd=40))
UNCONN 0      0            0.0.0.0:7001       0.0.0.0:*    users:(("python3",pid=63790,fd=40))
```

위 항목들을 x 가 정수가 되는 도메인 ID 와 participantId 를 계산하여 다음과 같이 구하였다.

```
12410 = 7400 + x * 250 + 2 * ? + 0 (x = 20, ? = 5)

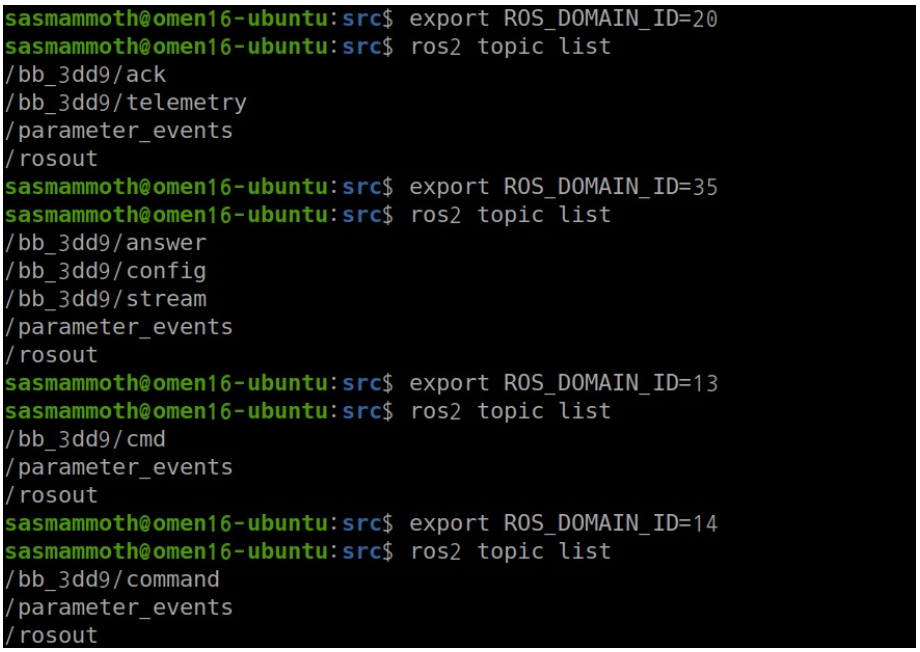
10910= 7400 + x * 250 + 2 * ? + 0 (x = 14, ? = 5)

10660 = 7400 + x * 250 + 2 * ? + 0 (x = 13, ? = 5)

16160 = 7400 + x * 250 + 2 * ? + 0 (x = 35, ? = 5)

7400 = 7400 + x * 250 + 2 * ? + 0 (x = 0, ? = 0)
```

이를 도메인 id 를 바꿔가며 토픽 리스트를 뽑아서 RTPS 식에 맞게 포트가 배정됨을 보였다.



8.3 궁금증

2 가지 궁금한이 있다.

1. 딱히 도메인 id 를 0 으로 하고 뭔가를 pub 한적이 없는데 왜 저게 존재하는가.
2. participantId 는 어떻게 해야 증가하는가? (의도치 않은 시도를 하였을때와 현재 자유탐구를 하였을 때 꽤 오랜 시간이 지났지만 둘다 participantId 가 5 이다.)